5 주차 학습노트

2022-16684 안휘준

지난 학습 요약 (1~4 주차)

인공지능이 만든 '우주 침략자' 게임 RTL 을 직접 빌드하고 검증하면서, AI 의 답과 "빌드 성공"은 도구로 다시 확인해야 한다는 것을 매주 확인했다.

2 주차	diff 로 Q.09 버전 비교, Gemini 가 만든 Makefile 오류 5 개 수정, 3x8 디코더 신경망 학습	AI 가 바꾼 것은 대부분 주석이었다. 빌드가 성공해도 실행해 봐야 숨은 문제(SystemC 경로)가 드러난다.
3 주차	Q.10 스프라이트 ROM 확인, Q.11 화살표 키 이상 반응 원인 분석, Q.13 시뮬레이션	키 문제는 테스트벤치의 느린 폴링과 RTL 의 클럭마다 이동, 두 원인이었다. 최종 RTL 은 STATE_CHK_BTN, btn_fire_pe, player_hit 로 고쳤다.
4 주차	Q.14 AI 면적 예측 검증(Yosys + ETRI 셀), emulation 3 단계 빌드, 꺼 둔 경고 다시 켜기	AI 는 면적 80% 감소를 예상했지만 실측은 0.8%였다. -Wno-WIDTHTRUNC 를 빼자 7'd128 잘림 버그(목숨 아이콘 누락)가 나왔다.

Q.13 실습 개요

같은 RTL(4_space_invaders_engine_glcd.v.Ok)을 시뮬레이션, 에뮬레이션, 프로토타이핑 세 방식으로 실행했고, 세 단계 모두 게임이 동작했다. 사용한 킷은 Cyclone IV E(EP4CE6E22C8) FPGA 보드(PSCE-TRANS)와 Raspberry Pi Pico 보드(PSCE-MI)이고, 두 USB 장치(JTAG FT232H, Pico)는 usbipd 로 WSL2(Ubuntu 24.04)에 연결했다.

항목	시뮬레이션	에뮬레이션 (CA)	프로토타이핑
회로가 도는 곳	PC (Verilator)	FPGA	FPGA
클럭 공급	SystemC (가상 10 MHz)	PC 가 USB 로 한 클럭씩	Pico PWM 100 kHz

항목	시뮬레이션	에뮬레이션 (CA)	프로토타이핑
화면	PC SDL 창	PC SDL 창 (같은 테스트벤치)	보드 OLED
한 화면 시간	—	실측 7.79 s (약 1.05 kHz)	81.92 ms (계산)
볼 수 있는 신호	모든 내부 신호	포트 9 개 (VCD)	핀 레벨 (계측기)
이번에 확인한 것	3 번 RTL 도 정상 동작	합성 가능한 RTL 만 올라감	계측 장비의 대역폭 한계

에뮬레이션은 45 프레임을 재서 평균 7.789 s(최소 7.033, 최대 10.723 s)가 나왔다. 8,192 클럭 ÷ 7.789 s ≈ 1,052 Hz 이므로 클럭 한 번에 USB 왕복이 약 0.95 ms 걸리고, 프로토타이핑보다 약 95 배 느리다.

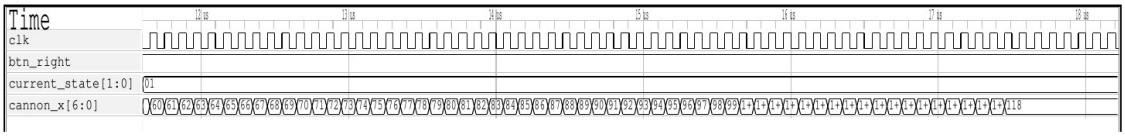
보드 실습에서 막힌 곳

단계	결과	문제와 해결
openFPGALoader	해결	설치돼 있지 않았고, apt 버전은 칩 ID(0x020f10dd)를 몰랐다. 최신 소스를 빌드해 설치했다.
에뮬레이션 TB 빌드	해결	git pull 로 sc_glcd128x64_TLM.h 가 바뀌었는데 Makefile 의존 목록(SC_HDRS)에 없어 다시 빌드되지 않았다. make -B 로 강제 빌드했다.
Pico 업로드	우회	BOOTSEL 로 재시작한 Pico 드라이브가 Windows 에만 보여 No drive to deploy 로 실패했다. .uf2 를 드라이브에 직접 복사했다.
FPGA 구성	성공	Load SRAM: 100.00%, Done. sudo 는 TTY 가 있는 터미널에서 직접 실행해야 했다.
에뮬레이션 실행	성공	Connection established..., SDL 에 게임 화면. Frame[0]~[44] 기록.
프로토타이핑 실행	성공	OLED 에 게임이 표시됐다.

하드웨어 단계의 실패는 모두 RTL 바깥(도구 설치, 빌드 의존성, USB 경로, 권한)에서 났다.

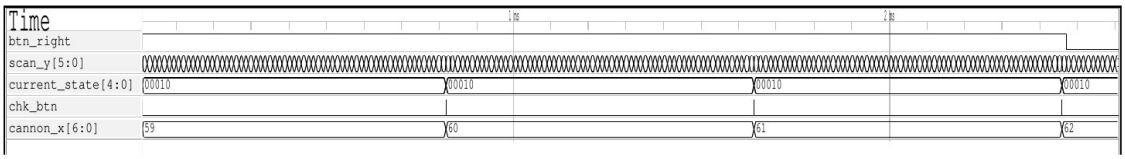
과제 8 — 버튼 확인을 화면 갱신에 맞춰야 하는 이유

0 번 RTL 은 버튼이 눌려 있는 동안 매 클럭 cannon_x 를 1 씩 바꿔서, 대포가 59 클럭 만에 화면 끝(118)에 닿는다. 클럭은 50 MHz 에서 20 ns 이고 사람이 키를 가장 짧게 눌러도 50~100 ms 이므로, 한 번 누르면 항상 끝까지 튀다. 화면을 그리는 도중에 좌표가 바뀌면 한 화면 안에 서로 다른 위치가 섞여 그려지는 문제도 있다.



W1. 0번 RTL(Org_wild_cannon): btn_right=1 이후 매 클럭 cannon_x가 1씩 증가 → 59에서 118까지 59클럭(5.9 µs @10 MHz). 헤드리스 TB, v0_hold1000.vcd, 11.0–18.2 µs

대책 (4 번 RTL): 스캔이 마지막 픽셀(127, 63)에 닿으면 STATE_CHK_BTN 을 1 클럭 거치고, 이때만 버튼을 확인한다. 대포는 한 화면(8,192 클럭)에 1 픽셀씩 움직인다.



W2. 4번 RTL(Ok): btn_right를 3프레임 유지. 화면 끝(scan_y 63→0)마다 CHK_BTN을 거쳐 chk_btn이 1클럭 뜨고 cannon_x가 59→60→61→62로 1씩 증가. v4_hold3frames.vcd, 0–2.6 ms

버튼을 누른 시간	0 번 RTL	4 번 RTL
1,000 클럭	59 클럭 뒤 끝(118) 도달	움직이지 않음 (프레임 경계를 안 지남)
3 프레임 (24,576 클럭)	—	59 → 62, chk_btn 3 회
1,000,000 클럭	—	483,217 클럭 뒤 118 도달 (≈ 59 × 8,192)

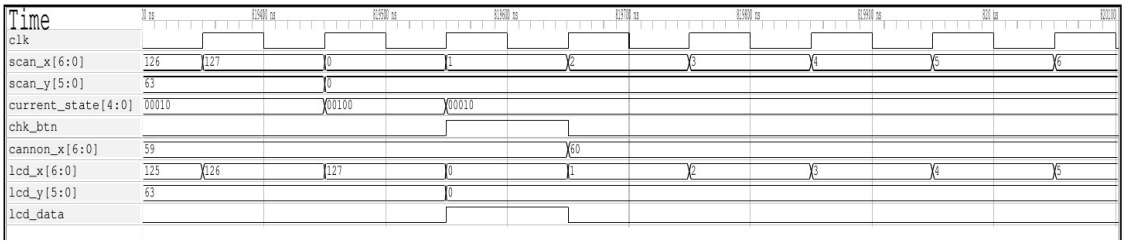
한계도 있다. 속도가 "프레임당 1 픽셀"이라 클럭이 빨라지면 대포도 빨라진다(10 MHz 에서 초당 1,220 픽셀, 100 kHz 에서 약 12 픽셀). 이동을 game_tick(시간 기준)에 묶거나, 동기화 → 디바운스 → 엣지 검출을 거치는 방법이 더 낫다.

과제 9 — V_Sync 가 필요한 이유

V_Sync 는 한 화면이 끝났다는 기준점이다. 그 덕분에 다음이 가능하다.

- 디스플레이와 게임 회로가 같은 화면 단위로 맞춰 움직인다.
- 화면을 그리지 않는 구간에 게임 상태를 한꺼번에 바꿔 화면 찢김(tearing)을 막는다.
- 일정한 갱신 주기가 게임 속도의 시간 기준이 된다.

이 설계에는 V_Sync 핀이 없다. 대신 scan_x==127 && scan_y==63 조건과 그 직후의 chk_btn 1 클럭 펄스가 그 역할을 한다. 테스트벤치(SDL_RenderPresent)와 Pico 펌웨어(bUpdateBuffer)도 같은 좌표에서 화면을 갱신한다.



W3. 프레임 경계(V_Sync 역할): scan_x=127, scan_y=63 → current_state=CHK_BTN(00100) → 다음 클럭 chk_btn=1 → cannon_x+1.
lcd_x/lcd_y는 scan보다 1클럭 늦음. 819.3–820.1 µs

W3 에서 부작용도 찾았다. STATE_CHK_BTN 인 1 클럭 동안 lcd_data 가 테두리를 그리는 분기를 타서, 좌상단 (0, 0) 픽셀이 매 프레임 1 로 그려진다. 또 총알과 침략자는 game_tick(16,001 클럭)에 맞춰 움직이는데 이 주기가 프레임(8,192 클럭)과 맞지 않아, 화면 도중에 좌표가 바뀔 수 있다(추정).

과제 10 — 시뮬레이션만으로 부족한 이유

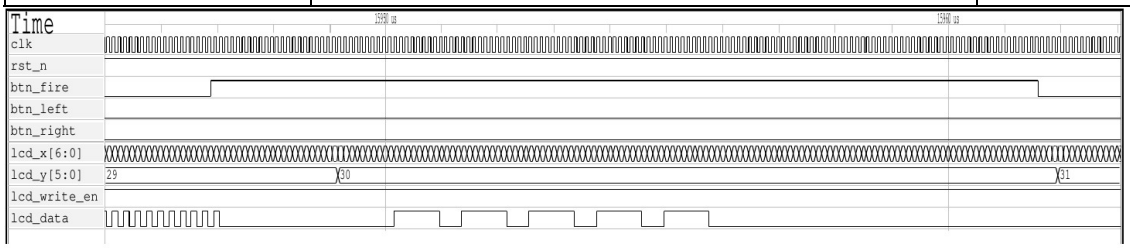
시뮬레이터는 언어의 실행 규칙을 따르고 합성기는 같은 코드를 실제 회로로 바꾼다. 두 해석이 다를 수 있어서 하드웨어로 확인하는 단계가 필요하다. 3 번 RTL 은 current_state 를 두 always 블록에서 동시에 대입하는데, 두 도구의 판단이 갈렸다.

도구	결과
Verilator (--lint-only -Wall)	경고 없음. 피격을 강제로 넣으면 정상적으로 DIE 상태가 된다.
Quartus	Error (10028): Can't resolve multiple constant drivers for net "current_state...". 합성 실패. 0 번도 같은 이유로 실패하고, 4 번은 오류 0 개.

시뮬레이션에서는 나중에 실행된 대입이 이겨서 우연히 맞게 동작했을 뿐이다. 실제 레지스터에는 입력을 두 개 연결할 수 없다. 4 번은 player_hit 플래그로 구동하는 곳을 하나로 모았다.
에뮬레이션은 이 차이를 걸러내면서도 시뮬레이션과 같은 테스트벤치를 재사용하므로 검증 결과를 그대로 비교할 수 있다.

과제 11 — 내부 신호 추적 문제

구분	방식	볼 수 있는 범위
Synopsys ZeBu	Verdi 와 통합. 전체 파형을 올리거나 일부만 캐프처한 뒤 재구성	설계 전체
Cadence Palladium	FullVision, Dynamic Probes, Record and Replay	모든 넷
Signal Tap / ILA	미리 고른 신호를 FPGA 내부 RAM 에 기록. 신호를 바꾸면 다시 합성	선택한 신호만
PSCE 킷	wrapper 가 DUT 포트만 읽어 VCD 로 남김	포트 9 개뿐



W4. 에뮬레이션 VCD(Espace_invaders_engine_glcd.vcd): FPGA에서 동작한 DUT의 포트 신호 9개만 기록됨(current_state, cannon_x 같은 내부 신호는 없음). Space 키 입력으로 btn_fire가 15,946.9–15,961.6 µs(147클럭) 동안 1, lcd_x는 매 클럭 증가. 시뮬레이션 시간 15,945–15,963 µs

PSCE 에서 내부 신호를 보려면 디버그 포트를 추가하고 wrapper 가 주고받는 바이트(NUM_OUT_ARRAY, N_TX/N_RX)를 늘려야 한다. 그만큼 클럭마다 통신량이 늘어 더 느려진다. 출처: Synopsys ZeBu Server 5 사양서, Cadence Palladium Z3 데이터시트, Intel Signal Tap 문서, AMD UG908(ILA).

과제 12 — 하드웨어 분석

(a) FPGA 부품과 핀 배치. 합성 스크립트의 부품명은 Cyclone IV E EP4CE6E144 이다. 두 모드의 tcl 핀 배치를 Pico 펌웨어 두 개의 GPIO 정의, 회로도와 대조했고 24 핀 모두 일치했다. 이 과정에서 같은 배선의 방향이 모드마다 반대인 핀 11 개를 찾았다. 예를 들어 GP28/PIN_135 는 에뮬레이션에서는 FPGA 출력, 프로토타이핑에서는 Pico 가 내보내는 클럭이다. 그래서 모드를 바꿀 때 펌웨어 업로드와 FPGA 구성을 연달아 실행했다.

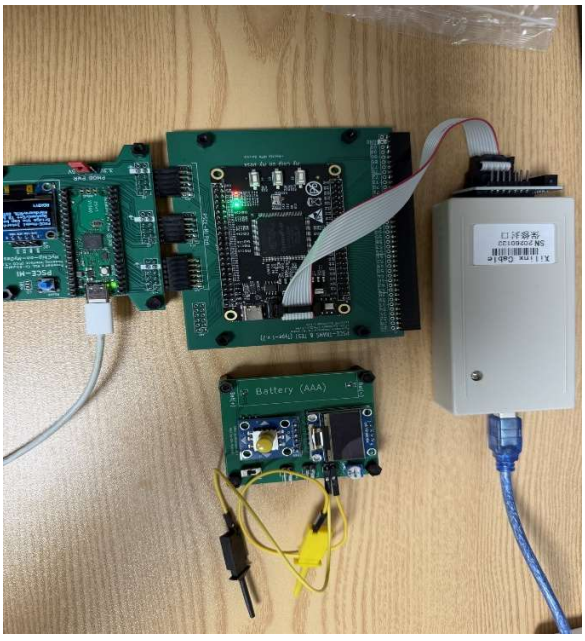
(b) 펌웨어 비교

항목	에뮬레이션 (MI)	프로토타이핑 (SA)
클럭	패킷을 받을 때마다 소프트웨어로 한 번 토글 (GP20 → PIN_24)	하드웨어 PWM 100 kHz, 듀티 50% (GP28 → PIN_135)
통신	매 클럭 입력 1 바이트를 보내고 출력 2 바이트를 받는 블로킹 왕복	버튼 상태 1 바이트를 보내기만 함
화면	PC 의 SDL 창	보드 OLED
속도	실측 약 1.05 kHz	100 kHz

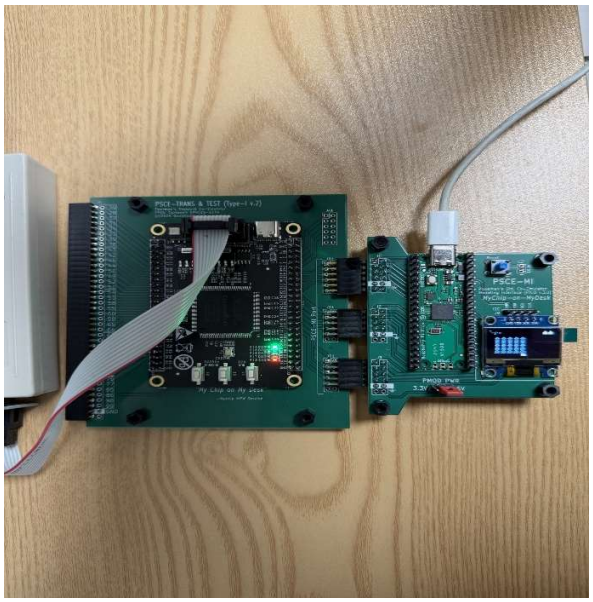
과제 13 — 미니 오실로스코프 측정

프로토타이핑 동작 중 1 주차에 실습한 ATmega328P 미니 스코프로 측정했지만, 100 kHz 클럭은 제대로 보이지 않았다.

스코프의 가장 빠른 시간축(200 µs/div)에서 샘플 간격은 8 µs(125 kS/s)이다. 이 속도로는 62.5 kHz 를 넘는 신호를 나타낼 수 없다. 100 kHz 클럭은 한 주기에 샘플이 1.25 개뿐이다.버튼 신호는 느린 레벨 변화라 원리상 측정할 수 있어야 하지만, 이번에는 트리거와 시간축 설정을 맞추지 못했다.



나눠 받은 보드(PSCE-TRANS + PSCE-MI)로 에뮬레이션과 프로토타이핑을 모두 동작시켰다.



에뮬레이션에서는 게임 화면이 PC 의 SDL 창에 그려지고, 보드 OLED 는 시작 문구("PSCE-Model Interface ... READY!")에 머문다.
코드상으로는 OLED 에도 화면이 미러링돼야 하므로, `io_req`(GP22) 인터럽트 경로를 원인으로 의심하고 있다(미확인).

프로토타이핑에서는 Pico 가 100 kHz 클럭을 주고 FPGA 가 혼자 돌며, 게임 화면이 보드 OLED 에 실시간으로 표시됐다.

느낀 점

에뮬레이션은 한 화면에 7.8 초가 걸릴 만큼 느렸지만, 시뮬레이션과 같은 테스트벤치로 "합성된 실제 회로"를 확인할 수 있었다. 프로토타이핑은 빠르지만 안을 들여다볼 방법이 거의 없었다. 3 번 RTL 처럼 시뮬레이션에서 멀쩡한 코드가 합성에서 막히는 것을 보면서, 칩을 만들기 전에 세 단계를 차례로 거쳐야 하는 이유를 실감했다.

하드웨어 단계에서 막힌 곳은 모두 RTL 바깥이었다. 도구 버전, 빌드 의존성, USB 경로, 권한, 계측기 대역폭은 시뮬레이션으로 미리 잡을 수 없는 문제다. 특히 오실로스코프는 재기 전에 장비가 대상을 잴 수 있는지부터 따져야 한다는 것을 배웠다. 2~4 주차에 이어 이번에도 "통과했다"는 말은 어느 도구가 무엇을 확인했는지까지 봐야 의미가 있었다.